Autoregressive and Adversarial Generative Models for MNIST

Name: 刘纳多 (LIU Naduo) Student ID: 60777841

Code and data (GitHub, public): https://github.com/Minnesinger02/CS5494AS1

CS5494 Topics in Generative AI — Assignment, November 2026

Training data: the entire 60,000-image MNIST training set (binarised for FVSBN and MADE; grayscale for the GAN)

Abstract

This report implements the three generators required by the assignment: a fully visible sigmoid belief network, a neural autoregressive model, and a generative adversarial network on MNIST. All autoregressive models are trained on 28×28 binarised digits and evaluated by exact held-out likelihood, reported both as nats per image and as bits per pixel so that results at different resolutions remain comparable. The generative adversarial network is trained on the original grayscale digits and evaluated with an independent classifier. Beyond the baseline implementations we report a validation-based hyperparameter study of the autoregressive model, a three-member mixture of autoregressive models, a convolutional adversarial architecture with a two-timescale update rule, and an exponential moving average of the generator weights. The final autoregressive model reaches 0.1223 bits per pixel as a three-member mixture (0.1260 for a single model), and the convolutional generator reaches a feature-Fréchet distance of 7.12 against 34.61 for the initial fully connected baseline.

Contents

1 Introduction **3**

2 Data and preprocessing **3**

3 Methods **3**
- 3.1 Fully visible sigmoid belief network . . . 3
- 3.2 Neural autoregressive model . . . 4
  - 3.2.1 Mixtures of autoregressive models . . . 4
- 3.3 Generative adversarial network . . . 4
  - 3.3.1 Fully connected baseline . . . 5
  - 3.3.2 Convolutional architecture . . . 5
  - 3.3.3 Two-timescale update rule . . . 5
  - 3.3.4 Exponential moving average of the generator . . . 5
- 3.4 Evaluation protocol . . . 5

4 Experiments **6**
- 4.1 Setup . . . 6
- 4.2 Autoregressive models . . . 6
- 4.3 Adversarial models . . . 9
- 4.4 Hyperparameter analysis . . . 11
- 4.5 Per-digit showcase . . . 12
- 4.6 Run time . . . 12

5 Discussion **12**

6 Conclusion **13**

A Run instructions **14**

B Code map and additional figures **14**

1 Introduction

The assignment asks for three separate image generators on MNIST (LeCun et al. 1998). Section 1.1 requires a *fully visible* sigmoid belief network, that is, a model with no latent or hidden variables whose conditionals are logistic; Section 1.2 requires a *neural* autoregressive model with one or more layers; Section 1.3 requires a generative adversarial network trained on the original, non-binarised digits, together with a short discussion of training stability. The two autoregressive models are trained on binarised digits, the generative adversarial network on grayscale digits, and each model is evaluated on the metric that its formulation supports: exact likelihood for the autoregressive models and a classifier-based distribution distance for the adversarial one.

The report makes four additions to a straightforward implementation. First, the autoregressive inputs use the full 28×28 resolution rather than a reduced grid, which lowers the cost per pixel by roughly a factor of two even though the total per-image likelihood rises. Second, the capacity, depth and regularisation of the autoregressive network were selected on a held-out slice of the *training* set, so the test set is read only once. Third, three independently seeded autoregressive models are combined into a mixture, and the effect of the mixture size is measured rather than assumed. Fourth, the adversarial model is first trained as a fully connected baseline, then as a convolutional network with a two-timescale update rule, and finally with an exponential moving average of the generator weights; each change is reported with the same three external statistics, which makes the improvement traceable rather than anecdotal.

2 Data and preprocessing

MNIST contains 60,000 training and 10,000 test images of size 28×28 with pixel intensities $v \in \{0, \dots, 255\}$. For the two autoregressive models the digits are binarised with a fixed threshold,

$$x_{ij} = \mathbf{1}\left[\frac{v_{ij}}{255} \geq 0.5\right], \tag{1}$$

which yields a mean ink coverage of 13.2% over the training set; a pixel is therefore much more likely to be background than stroke, and a model that only predicts the marginal distribution already reaches 0.5636 bits per pixel. No resizing is applied, so the autoregressive input is a 784-dimensional binary vector in raster order. A 14×14 variant with the same threshold is included as a resolution reference in Section 5 and stored under `results/ablation_14x14`. For the generative adversarial network the digits keep their grayscale values and are scaled to $[-1, 1]$, which matches the tanh output of the generator.

The training subset is drawn once with `torch.randperm` under seed 42 and stored with each run, so every experiment that uses the same number of images sees exactly the same digits. The $10{,}000$-image ablation reuses the first $10{,}000$ entries of that permutation. Hyperparameter selection used a further split of $5{,}000$ images taken from the *front* of the training subset; those images were never used for gradient updates while tuning, and the final models are retrained on all $60{,}000$ images with the selected settings. The test set is used exclusively for the numbers reported in Section 4.

3 Methods

3.1 Fully visible sigmoid belief network

The fully visible sigmoid belief network factorises the joint distribution into one logistic conditional per pixel,

$$p(\mathbf{x}) = \prod_{i=1}^{D} p(x_i \mid x_1, \dots, x_{i-1}), \qquad p(x_i = 1 \mid \mathbf{x}_{<i}) = \sigma\left(b_i + \sum_{j<i} W_{ij} x_j\right), \tag{2}$$

with $D = 784$ pixels in raster order and σ the logistic function. Because there are no hidden units, the model is exactly a chain of logistic regressions. Causality is enforced structurally: the weight matrix is

multiplied elementwise by a mask

$$M_{ij} = \mathbf{1}[\, j < i \,], \tag{3}$$

so output i can never receive information from pixel i or from any later pixel. Training minimises the negative log-likelihood of the training set,

$$\mathcal{L}(\mathbf{x}) = -\sum_{i=1}^{D} \Big[x_i \log \sigma(z_i) + (1 - x_i) \log \big(1 - \sigma(z_i)\big) \Big], \tag{4}$$

which is the sum of the 784 per-pixel binary cross entropies; this is also the quantity reported as nats per image. Sampling starts from an empty canvas and draws one pixel at a time in raster order, using the model's own predictions as the conditioning context.

3.2 Neural autoregressive model

The second model keeps the same factorisation but replaces the logistic conditional with a masked multi-layer network, following the masked autoencoder for distribution estimation (Germain et al. 2015). Every input pixel, every hidden unit and every output unit is assigned a *degree*; information may only flow from lower to higher degrees. With input degrees $d^{\text{in}}_j = j$ and hidden degrees $d^{\text{hid}}_k = k \bmod (D - 1)$, the three masked layers satisfy

$$d^{\text{in}}_j \le d^{\text{hid}}_k, \qquad d^{\text{hid}}_k \le d^{\text{hid}}_{k'}, \qquad d^{\text{hid}}_k < d^{\text{out}}_i, \tag{5}$$

where the middle condition applies between consecutive hidden layers. Any path from an input pixel to output i is a chain of non-decreasing degrees that ends with a strict inequality, hence it can only start at a pixel $j < i$; output i therefore never depends on the current or a future pixel. The implementation uses one hidden layer of 1536 rectified linear units, which gives about 2.41 million parameters, and the same summed binary cross entropy as in Equation 4. Sampling is sequential as well: 784 forward passes per image, which is why the measured sampling time is two orders of magnitude larger than for the adversarial generator.

3.2.1 Mixtures of autoregressive models

Independent runs of the same architecture differ in initialisation and in the order in which mini-batches are presented, and their errors are only partly correlated. A mixture of K models is itself a normalised density,

$$p_{\text{mix}}(\mathbf{x}) = \frac{1}{K} \sum_{k=1}^{K} p_k(\mathbf{x}), \qquad \log p_{\text{mix}}(\mathbf{x}) = \operatorname{logsumexp}_k \log p_k(\mathbf{x}) - \log K, \tag{6}$$

so its held-out likelihood can be compared directly with a single model. Averaging the logits and applying a sigmoid would instead produce a geometric mean, which is not normalised and therefore not a valid likelihood; that shortcut is deliberately not used. The practical cost is linear in K: K times the parameters, K times the evaluation time, and no closed-form way of sampling from the mixture of autoregressive models, so generated images are drawn from one member and this is stated wherever such images appear.

3.3 Generative adversarial network

The generator maps a 64-dimensional standard normal vector to an image and the discriminator maps an image to a single logit; both are trained with the non-saturating objective of (Goodfellow et al. 2014),

$$\min_{G} \max_{D} \; \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \big[\log D(\mathbf{x}) \big] + \mathbb{E}_{\mathbf{z}} \big[\log \big(1 - D(G(\mathbf{z}))\big) \big]. \tag{7}$$

Each mini-batch performs one discriminator step on real and detached fake images, followed by one generator step.

3.3.1 Fully connected baseline

The first architecture follows the same spirit as the original formulation: the generator maps $64 \to 256 \to 512 \to 784$ with leaky ReLU slopes of 0.2 and a tanh output, and the discriminator maps $784 \to 512 \to 256 \to 1$. It is simple to train but has no notion of locality, and its samples are visibly noisier than the convolutional ones (Section 4.3).

3.3.2 Convolutional architecture

The second architecture is a DCGAN-style pair (Radford et al. 2016). The generator projects the latent vector to $128 \times 7 \times 7$, applies batch normalisation (Ioffe and Szegedy 2015) and two 4×4 transposed convolutions with stride 2, reaching 28×28 with a tanh output. The discriminator mirrors this with two strided 4×4 convolutions and a linear logit. Batch normalisation stabilises both players but also makes the discriminator's statistics depend on the batch, which interacts with the learning-rate choice below.

3.3.3 Two-timescale update rule

With mini-batches of 512 images the discriminator receives a comparatively low-variance gradient estimate, so at a shared learning rate it overtakes the generator within a few epochs and the generator loss drifts upwards. Following the two-timescale update rule (Heusel et al. 2017) we keep the generator at 2×10^{-4} and lower the discriminator to 2×10^{-5}. The effect is large and is measured rather than asserted: with the shared rate the same architecture reaches a feature distance of 38.30, whereas the two-timescale version reaches 7.71 with the fully connected pair and 7.15 with the convolutional pair (Table 3).

3.3.4 Exponential moving average of the generator

Adversarial training does not minimise a fixed objective, so the generator oscillates around the equilibrium and the last iterate is not necessarily the best one. We therefore keep a shadow copy of the generator parameters,

$$\theta_{\text{ema}} \leftarrow d\,\theta_{\text{ema}} + (1 - d)\,\theta, \qquad d = 0.999, \tag{8}$$

updated after every generator step. Only floating-point parameters are averaged; the batch normalisation running statistics are copied from the live model when the checkpoint is written, because averaging them would mix statistics from different weight configurations. The average is used only for sampling: training, previews and the reported loss curves all use the live weights, so the training curves remain comparable with the run without averaging. The checkpoint is stored alongside the conventional one, which allows the two to be evaluated under identical conditions.

3.4 Evaluation protocol

The autoregressive models are scored by the negative log-likelihood of the $10{,}000$ test images, in nats per image, and converted to bits per pixel,

$$\text{bpp} = \frac{\text{NLL}}{D \ln 2}, \tag{9}$$

with $D = 784$. The conversion matters because nats per image grows with the number of pixels: a 14×14 model with 33.66 nats is worse per pixel (0.2478 bpp) than a 28×28 model with 68.59 nats (0.1262 bpp). All cross-resolution statements in this report use bits per pixel.

The adversarial generator has no tractable likelihood, so it is scored with an independent convolutional classifier trained on real MNIST (test accuracy 96.4%). Three statistics are reported: the mean maximum class probability (confidence), the entropy of the predicted-class histogram over $2{,}000$ samples (maximum $\ln 10 = 2.303$), and a Fréchet distance between the 128-dimensional penultimate features of $2{,}000$ generated and $5{,}000$ real images,

$$\text{FD} = \lVert \mu_g - \mu_r \rVert^2 + \operatorname{tr}\!\left(\Sigma_g + \Sigma_r - 2(\Sigma_g \Sigma_r)^{1/2} \right). \tag{10}$$

Real MNIST scores confidence 0.939, entropy 2.299 and FD $= 0$ by construction; a self-comparison of two real batches gives FD $= 0.02$, which indicates the numerical noise floor. The measure is a proxy, not the standard FID, and differences below about one unit should not be treated as meaningful with $2{,}000$ samples.

The same classifier serves as the labelling device for every sample figure in this report. Because all generators are unconditional, $4{,}000$ unconditional samples are drawn per model, labelled by the classifier, and the first ten samples assigned to each digit are kept. Columns of those figures therefore correspond to the digits $0, 1, \ldots, 9$ and every row is an independent draw, so a column shows ten different attempts at the same digit. Selection is by predicted class only — samples are neither ranked by confidence nor chosen for appearance — and the number of candidates needed is reported in Section 4.5.

4 Experiments

4.1 Setup

Table 1 lists the configuration of the reported runs. Every run uses seed 42 for data selection, and the three mixture members differ only in the seed used for weight initialisation and batch order.

Table 1: Configuration of the reported runs. The two autoregressive families share the optimiser and schedule; the adversarial models use the DCGAN betas.

	FVSBN	MADE	DCGAN
Input	28×28 binary	28×28 binary	28×28 grayscale in $[-1, 1]$
Epochs	400	400	400
Batch size	1024	1024	512
Learning rate	10^{-3} + cosine	10^{-3} + cosine	2×10^{-4} (G), 2×10^{-5} (D)
Adam betas	$(0.9, 0.999)$	$(0.9, 0.999)$	$(0.5, 0.999)$
Parameters	0.62 M	2.41 M	0.54 M (G) + 0.14 M (D)

4.2 Autoregressive models

Table 2: Test likelihood on all $10{,}000$ MNIST digits. Bits per pixel is comparable across resolutions; nats per image is not. The mixture rows combine independently seeded MADE models with Equation 6; their parameter count and evaluation cost grow with K.

Model	Train n	Test NLL	bits/px	Train (s)	100 samples (s)	Params
FVSBN	60,000	75.72	0.1393	47.4	0.064	0.62 M
MADE, single (mean of 3 seeds)	60,000	68.45 ± 0.08	0.1260	52.1	0.100	2.41 M
MADE mixture, $K = 2$	60,000	67.07	0.1234	—	0.100	4.82 M
MADE mixture, $K = 3$	60,000	**66.46**	**0.1223**	—	0.100	7.23 M
FVSBN	10,000	120.17	0.2211	12.7	0.071	0.62 M
MADE, single (mean of 3 seeds)	10,000	80.19 ± 0.13	0.1476	13.9	0.102	2.41 M
MADE mixture, $K = 3$	10,000	76.92	0.1415	—	0.102	7.23 M

Table 2 and Figures 1–2 summarise the autoregressive results; in both figures the samples are grouped by the digit predicted by the classifier, following the convention of Section 3.4. Three observations stand out.

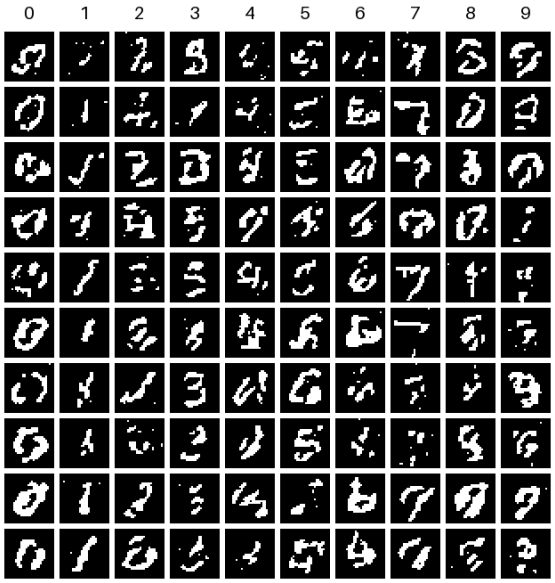
The linear model is a strong baseline. With 784 inputs the strictly lower-triangular logistic model reaches 0.1393 bits per pixel, which is already 4.0 times better than the marginal-distribution baseline (0.5636 bpp). Most of the remaining structure at this resolution is therefore additive in earlier pixels.

The nonlinear model still wins. MADE reduces the cost to 0.1260 bits per pixel, a 9.5% relative reduction, and the visible difference in Figures 1–2 is consistent with the number: the linear samples

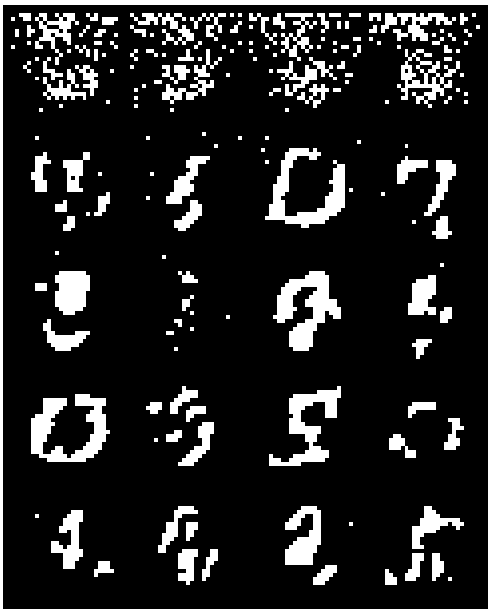
are fragmented, whereas the masked network produces continuous strokes. Figure 2 also shows that the model is not perfectly calibrated at the level of individual digits, but strokes are connected.

The mixture helps and is measurable. Going from one to three members lowers the test likelihood from 68.45 to 66.46 nats (2.9%), and the gain from $K = 1$ to $K = 2$ is already two thirds of the total (Figure 3). Because the members are trained independently, this gain is a genuine variance reduction rather than a longer training schedule; the price is three times the parameters and three times the evaluation cost, while sampling still uses a single member.

Every mixture member reproduces to within 0.01 nats from its saved weights when the likelihood is recomputed on the CPU; the residual is consistent with TF32 accumulation order, and the same check on a CPU-trained model reproduces to 10^{-7}.



(a) Samples grouped by predicted digit: columns $0 \ldots 9$, ten draws per column.



(b) Training stages 1, 101, 201, 301 and 400 (one stage per row).

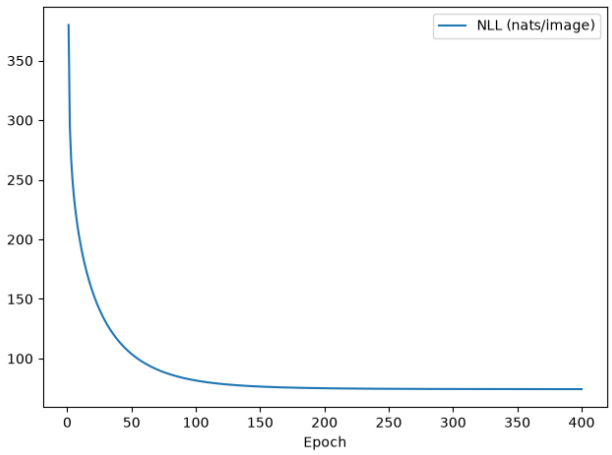
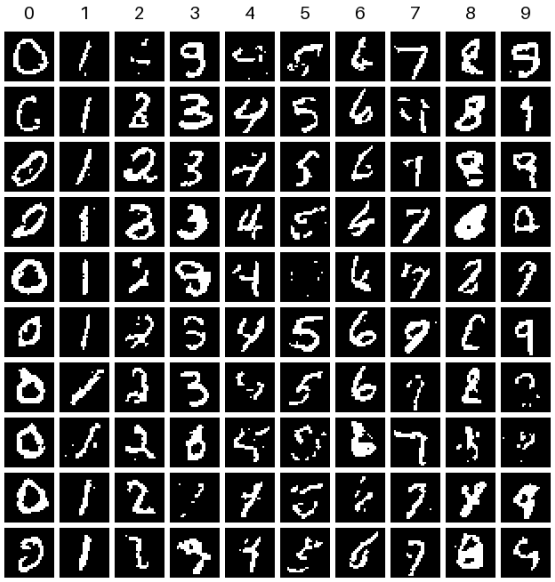
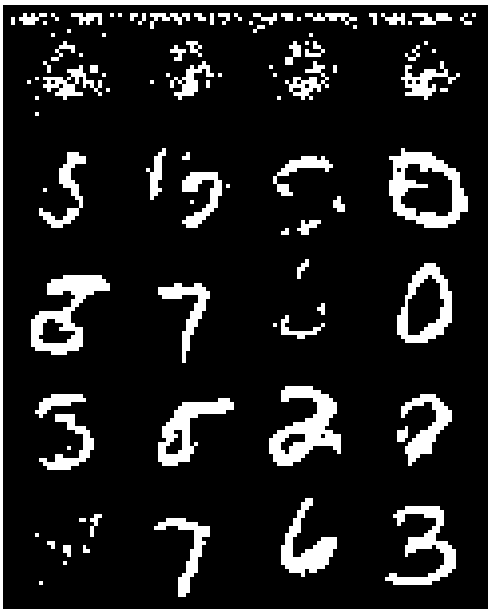


Figure 1: FVSBN: samples, training progress and training negative log-likelihood.



(a) Samples grouped by predicted digit, same layout as Figure 1.



(b) Training stages, same rows as Figure 1.

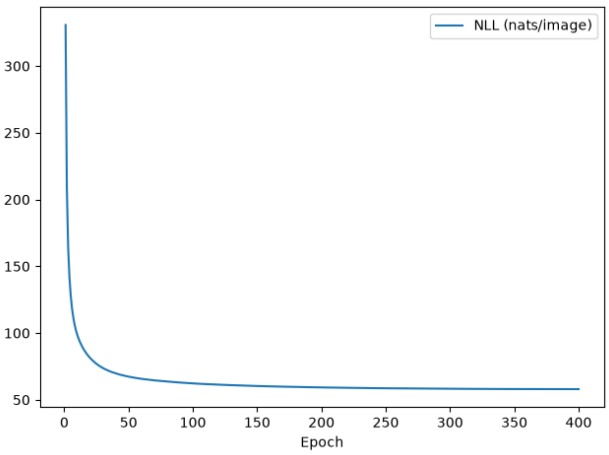


Figure 2: MADE: samples, training progress and training negative log-likelihood.

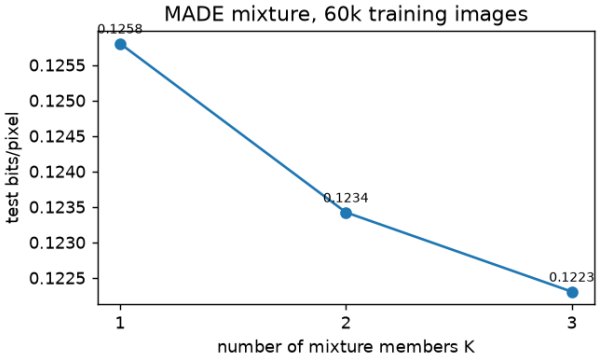


Figure 3: Test bits per pixel of the MADE mixture as a function of the number of members K on the same 10,000 test images.

4.3 Adversarial models

Table 3: Generative adversarial models, scored with an independent classifier. Confidence and entropy are computed from 2,000 samples; FD is a feature Fréchet distance to 5,000 real images (lower is better). Real MNIST: 0.939 / 2.299 / 0.

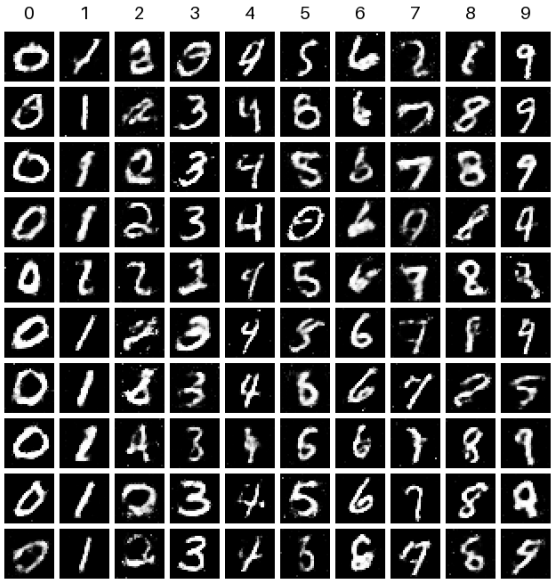
Model	Architecture	Epochs	Confidence	Entropy	FD	s/epoch
MLP, initial run	fully connected	60	0.8802	2.1737	34.61	5.29
MLP, shared learning rate	fully connected	150	0.8787	2.1655	38.30	0.89
MLP, two-timescale	fully connected	150	**0.8852**	2.2808	7.71	0.90
DCGAN, live weights	convolutional	400	0.8477	2.2735	7.15	0.80
DCGAN, moving average	convolutional	400	0.8476	2.2740	**7.12**	0.80

Figure 4 shows the fully connected baseline and Figure 5 the convolutional model; the appendix compares the three fully connected runs side by side. The first row of Table 3 is an initial run with a single learning rate at a batch size of 128. Increasing the batch while keeping that rate made the discriminator dominant and raised the feature distance to 38.30, which is the empirical reason for adopting the two-timescale rule. With the rule, the same architecture reaches 7.71, and the convolutional architecture reaches 7.15 with visibly smoother strokes.

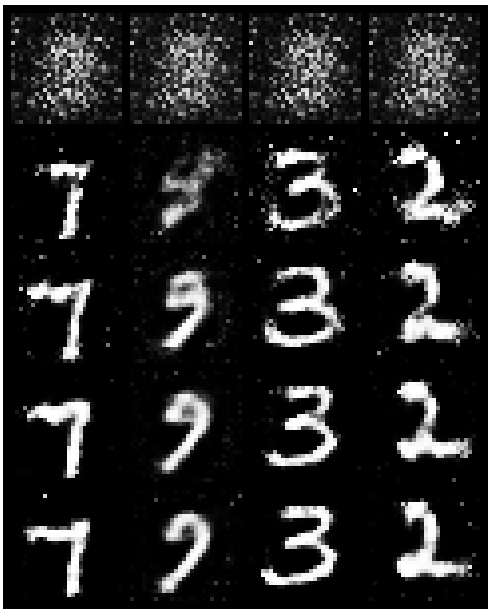
All five runs cover the ten digits, contain no blank images and produce class histograms without a dominant class, so there is no complete mode collapse. The smallest predicted-class share in 2,000 samples is 3.5% for the initial run and 6.2% for the convolutional one, while the largest is 19.3% and 15.5% respectively, so coverage is uneven rather than absent: 9 and 8 are drawn more often than 4 and 5, and the gap is visible in the digit columns of Figures 4–5.

The generator and discriminator losses of the convolutional run stay in a narrow band — about 0.93 and 1.12 at the end of training — which is the oscillation expected around the adversarial equilibrium rather than evidence of convergence; the loss panel of Figure 5 shows the same behaviour over all 400 epochs.

Using the moving average changes the checkpoint only; the training curve and the loss values are identical to the live-weight run. The three external statistics are also almost identical: confidence 0.8476 against 0.8477, entropy 2.2740 against 2.2735, and FD $= 7.12$ against 7.15. A difference of 0.03 is far below the run-to-run spread of this architecture, since two runs of the same configuration differ by 0.14 in FD (7.15 and 7.29 for two such runs) because cuDNN convolution kernels are not bit-reproducible. The moving average is therefore reported as a stabilising option that costs nothing at inference, not as a demonstrated quality gain; the sample grid is included so the reader can judge the visual difference independently.



(a) Samples grouped by predicted digit.



(b) Training stages with four fixed latent vectors per row.

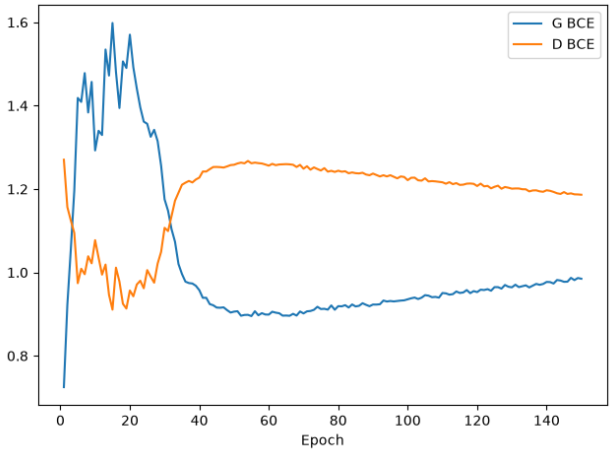
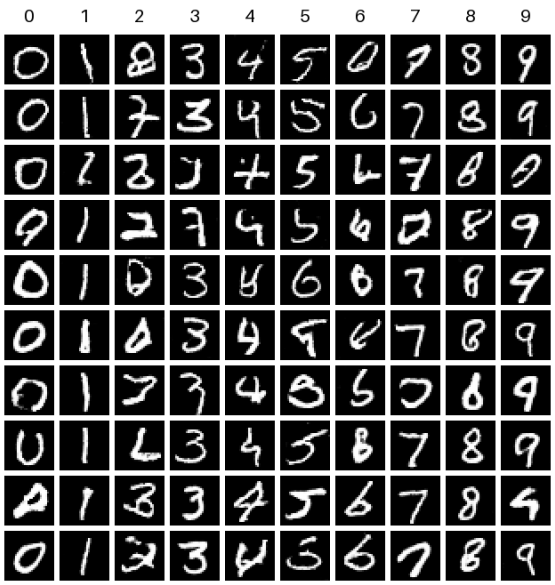
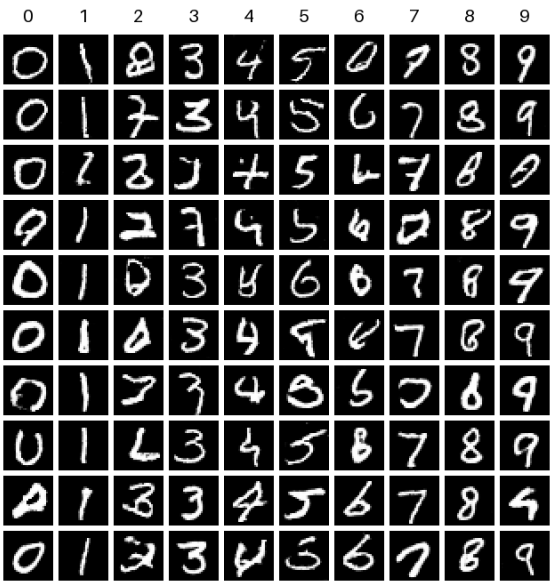


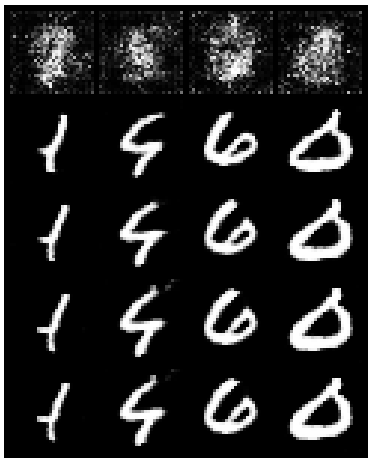
Figure 4: Fully connected baseline trained with the two-timescale rule.



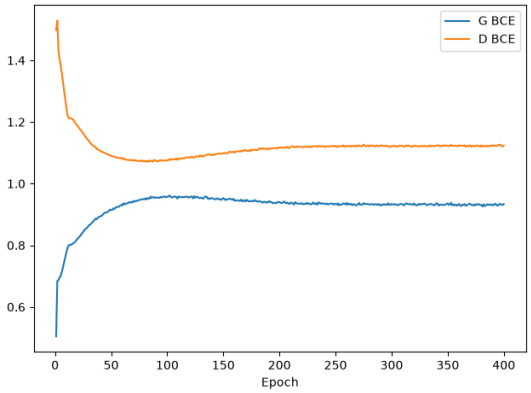
(a) Live weights.



(b) Moving average.



(c) Training stages.



(d) Generator and discriminator loss.

Figure 5: Convolutional generator: samples grouped by predicted digit for the live weights and for the moving average, the training-stage grid and the adversarial losses.

4.4 Hyperparameter analysis

Table 4: Masked autoregressive configurations. Selection used 5,000 images held out of the training split; the test column is shown only for reference and did not guide the choice.

Configuration	Validation bpp	Test bpp	Train (s)
1536 units, 1 layer, 400 epochs	**0.1287**	0.1266	44
2048 units, 1 layer, 400 epochs	0.1295	0.1276	44
1024 units, 1 layer, 400 epochs	0.1320	0.1296	69
1024 units, 1 layer, 800 epochs	0.1341	0.1318	86
1024 units, 2 layers, weight decay 10^{-3}	0.1376	0.1349	64
1024 units, 2 layers, 400 epochs	0.1443	0.1416	47
1024 units, 2 layers, weight decay 10^{-2}	0.1450	0.1419	75
1024 units, 2 layers, dropout 0.1	0.1469	0.1438	49
512 units, 1 layer, 400 epochs	0.1618	0.1589	45

Table 4 reports the nine most informative of fourteen configurations that were trained for this study. Three findings shaped the final model.

A single wide layer is consistently better than two narrower ones: at 1024 units the validation cost is 0.1320 against 0.1443 bits per pixel. With the mask conditions of Equation 5, every additional masked layer restricts the set of admissible paths, so depth buys little here and costs optimisation difficulty. Dropout and weight decay did not help at this data size, which suggests that the gap between training and test likelihood is not the binding constraint. Doubling the number of epochs from 400 to 800 made the validation cost worse ($0.1320 \to 0.1341$), so the cosine schedule at 400 epochs was kept.

4.5 Per-digit showcase

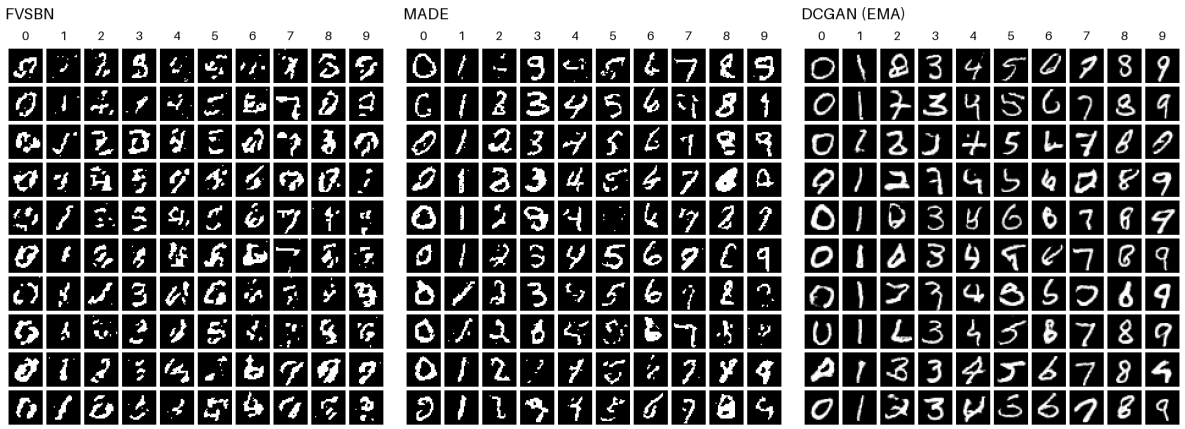


Figure 6: Unconditional samples grouped by the digit predicted by the independent classifier, shown side by side for the linear model, the masked autoregressive model and the convolutional generator. Columns are $0, 1, \ldots, 9$; each row is an independent draw. The same convention is used in Figures 1–5.

The generators are unconditional and have no class input, so a digit-ordered grid cannot be produced by conditioning. As described in Section 3.4, $4{,}000$ unconditional samples are drawn per model, labelled by the classifier, and the first ten assigned to each digit are kept; samples are not ranked by confidence and are not selected for appearance. The pool size is reported because it shows that the selection does not hide a collapse: the rarest predicted class contains 52 of $4{,}000$ draws for the linear model but 249 for MADE and 243 for the convolutional generator with the moving average.

4.6 Run time

Training and sampling times are listed in Table 2 for the autoregressive models and Table 3 for the adversarial ones. The full pipeline on one RTX 5080 laptop GPU — three mixture members at $60{,}000$ images, their $10{,}000$-image counterparts, one adversarial model, sampling, evaluation and the digit grid — takes about ten minutes, of which the adversarial model accounts for roughly 60%. Autoregressive sampling is the slowest inference path because it performs 784 sequential forward passes, but a batch of 100 images still completes in about 0.1 s.

5 Discussion

Resolution. The 14×14 ablation shows why bits per pixel is the right unit. The 14×14 model reaches 33.66 nats per image, the 28×28 model 68.45; the first looks better until the pixel count is accounted for, at which point the full-resolution model is twice as efficient per pixel. The apparent paradox comes from area-downsampling: averaging four pixels and thresholding produces a denser, less predictable binary image, so each coarse pixel carries more entropy than each fine pixel.

Linear versus nonlinear. At 28×28 the linear model is only 9.5% worse than the masked network, while at 14×14 the gap was 17%. The nonlinear model therefore helps most where the binary resolution destroys information; at full resolution a large part of the remaining structure is still additive. This also explains why the mixture, rather than a larger network, was the cheapest way to improve the likelihood: widening from 1536 to 2048 units did nothing, while averaging three independently trained models removed 2 nats.

Adversarial stability. The three adversarial runs illustrate the interaction between batch size and learning rate. The initial run used 128-image batches with a shared rate and converged slowly but stably. Moving to 512-image batches without changing the rate reduced the noise in the discriminator gradient enough for it to dominate, and the measured distribution distance degraded beyond the initial run. Halving the discriminator rate restored balance, and the convolutional architecture then improved the distance further while producing smoother strokes. The moving average is a cheap additional stabiliser: it does not change training, only which weights are used at sampling time. Class histograms show no collapse, but the imbalance between frequent (9, 8) and rare (4, 5) shapes persists in every run, so coverage is uneven rather than absent.

Limitations. All results are single-seed except the autoregressive mixture, so no significance test is offered for the adversarial comparisons; the mixture spread (± 0.08 nats) is reported as a rough guide to seed noise. The Fréchet distance uses a classifier trained on MNIST rather than an external feature extractor, so it is a proxy and is treated as such. The binarisation threshold is fixed at 0.5; the dynamically binarised variant used in some of the literature is harder, and numbers should not be compared across the two protocols. Finally, all models are unconditional, so the digit grid of Section 4.5 is a selection rather than a demonstration of conditional control.

6 Conclusion

The three required generators were implemented, trained and evaluated with the metric that each formulation supports. On 28×28 binarised digits the linear fully visible model reaches 0.1393 bits per pixel and the masked autoregressive model 0.1260; mixing three independently seeded members lowers this to 0.1223 at three times the parameter count. On the original grayscale digits the convolutional generator reaches a feature distance of 7.12, against 34.61 for the fully connected initial run, with ten classes covered and no blank samples. The accompanying repository contains all checkpoints, per-run configurations, training histories, the scripts that reproduce every table and figure, and a single command that reruns the pipeline on a Windows machine with a CUDA GPU.

A Run instructions

The repository is self-contained: MNIST is read from the bundled `data/` folder, and no network access is needed. On Windows with the prepared virtual environment:

```
$py = 'D:\HKStudy\.venv\Scripts\python.exe'
& $py -m cs5494.checks units
& $py -m cs5494.train --models fvsbn made --samples 60000 --test-samples 10000 `
      --side 28 --hidden 1536 --layers 1 --epochs 400 --batch 1024 --lr 0.001 `
      --scheduler cosine --device cuda --workers 4 --seed 42 `
      --output results/autoregressive
& $py -m cs5494.train --models gan --samples 60000 --epochs 400 --batch 512 `
      --lr 0.0002 --lr-d 0.00002 --arch dcgan --scheduler cosine --ema 0.999 `
      --device cuda --workers 4 --output results/gan_ema
& $py -m cs5494.evaluate mixture --runs results/made_seed42 `
      results/made_seed43 results/made_seed44
& $py -m cs5494.evaluate gan --runs results/gan_ema --weights ema
& $py -m cs5494.figures digits
& $py -m cs5494.checks report
cd report; powershell -NoProfile -ExecutionPolicy Bypass -File build.ps1
```

The same commands work on Colab after installing `requirements.txt` and replacing the interpreter with `python`; `assignment.ipynb` wraps them in a notebook. The script `run_windows.ps1` runs the whole pipeline, including the checks and the comparison document. Every script comments the non-obvious parts, and each run writes a `config.json` that records the exact arguments, so any table entry can be traced back to the command that produced it.

B Code map and additional figures

Table 5: Files in the shared folder.

File	Responsibility
`cs5494/train.py`	models (FVSBN, MADE, MLP/DCGAN), training loop, previews, metrics
`cs5494/sample.py`	resample from a checkpoint, optionally the moving-average weights
`cs5494/evaluate.py`	classifier scores for GANs, mixture likelihood for MADE
`cs5494/figures.py`	per-digit grids and the MLP training-regime figure
`cs5494/checks.py`	unit tests, NLL re-computation, report/JSON consistency, hashes
`cs5494/compare_runs.py`	side-by-side comparison of result folders
`report/`	this document, `main.tex` + `refs.bib`

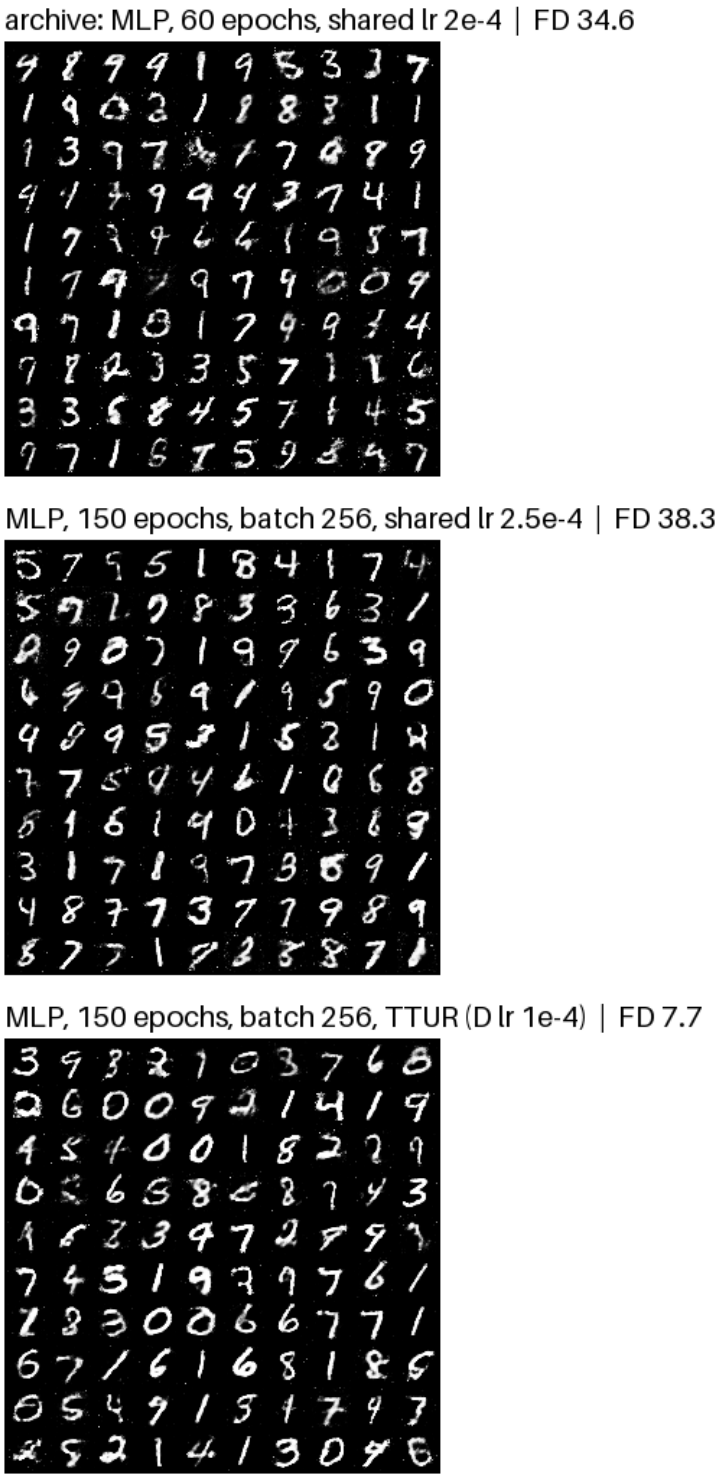


Figure 7: Fully connected generator under three training regimes. The first uses a shared learning rate, the second keeps that rate at a larger batch, and the third applies the two-timescale rule. Feature distances are 34.61, 38.30 and 7.71 respectively.

References

Germain, Mathieu et al. (2015). “MADE: Masked Autoencoder for Distribution Estimation”. In: *Proceedings of the 32nd International Conference on Machine Learning*, pp. 881–889.

Goodfellow, Ian et al. (2014). “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems 27*, pp. 2672–2680.

Heusel, Martin et al. (2017). “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium”. In: *Advances in Neural Information Processing Systems 30*, pp. 6626–6637.

Ioffe, Sergey and Christian Szegedy (2015). “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”. In: *Proceedings of the 32nd International Conference on Machine Learning*, pp. 448–456.

LeCun, Yann et al. (1998). “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.

Radford, Alec, Luke Metz, and Soumith Chintala (2016). “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks”. In: *International Conference on Learning Representations*.